RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 5

Project Definition

Total Marks: 10

Field	Detail
Institute	Red & White Skill Education
Subject	Deep Learning
Project	Practical Report 5 (PR 5) — Sentiment Analysis with RNN, LSTM and GRU
Total Marks	10
Project Duration	2 Days · 2 Hours per Day = 4 Hours
Dataset	IMDB Dataset of 50K Movie Reviews (Kaggle) — real, raw English text
Task Type	Binary text classification (positive / negative) — Many-to-One sequence model
Framework	TensorFlow 2.x / Keras, scikit-learn, pandas (Python 3.x, Jupyter Notebook)
Scope	SimpleRNN, LSTM and GRU only. Bidirectional layers, Attention, Transformers and pretrained embeddings (GloVe / Word2Vec / BERT) are out of scope for this practical.

Lecture Topic Coverage Map — Module 9

Task	Lecture Topics Covered (Module 9)
Task 1	Text as sequential data: cleaning, tokenisation, padding (preparation for every topic below)
Task 2	9.1 — What is an RNN, RNN intuition, WHY RNNs are needed (RNN vs ANN), forward propagation architecture in RNN
Task 3	9.2 — Sentiment Analysis with RNN, Types of RNN (one-to-one, one-to-many, many-to-one, many-to-many)
Task 4	9.3 — Backpropagation in RNN (BPTT) and problems with RNN (vanishing and exploding gradients)
Task 5	9.3 / 9.4 — LSTM (Long Short-Term Memory): cell state, forget / input / output gates
Task 6	9.5 — Gated Recurrent Unit (GRU) networks: update and reset gates
Task 7	Final comparison of RNN vs LSTM vs GRU, inference function and recommendation

§ 1 · Dataset

This project uses the IMDB Dataset of 50K Movie Reviews, a real-world text corpus originally assembled by Maas et al. (Stanford, 2011) and widely mirrored on Kaggle. Each row is a full-length, raw movie review written by a real user, labelled positive or negative. The reviews contain HTML fragments, mixed punctuation, negations and sarcasm, and range from a handful of words to over 2,000. Because the meaning of a review depends on word ORDER and long-range context ('not good at all' versus 'good'), this is exactly the kind of data a feed-forward ANN handles poorly and a recurrent network is designed for.

Field	Detail
Dataset Name	IMDB Dataset of 50K Movie Reviews
Kaggle URL	https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
File	IMDB Dataset.csv (single CSV, two columns)
Total Rows	50,000 reviews — 25,000 positive and 25,000 negative (balanced)
Known Data Issues	Contains HTML tags (for example), mixed case, punctuation and a few hundred exact duplicate reviews — all must be handled in Task 1
Review Length	Highly right-skewed: mean is roughly 230 words, median is lower, longest reviews exceed 2,000 words
Original Source	Maas et al., 2011, Stanford AI Lab (Large Movie Review Dataset)
Licence	Freely available for research and educational use — cite the original authors in your README

Dataset columns:

Column	Type	Description
review	*Text*	Raw review text. Contains HTML tags, upper/lower case, punctuation, numbers and emoticons.
sentiment	*Text*	Target label: 'positive' or 'negative'. Encode as positive = 1, negative = 0.

Note: Download IMDB Dataset.csv from the Kaggle link above and load it with pd.read_csv. Do NOT use keras.datasets.imdb in this practical — that version is already converted to integer IDs, and you are expected to do the text cleaning, tokenisation and padding yourself (Task 1). Use train_test_split with test_size=0.2, random_state=42 and stratify=y.

§ 2 · Project Tasks — What to Do

Complete ALL tasks in a single Jupyter Notebook named DL_PR5.ipynb. Begin every task with a Markdown cell containing: task title, one-sentence objective and expected output. Reuse your build function pattern from earlier practicals: write one build_model(cell_type, units, ...) function and change only one thing per experiment. Foundations from earlier practicals (Dense layers, Adam, binary cross-entropy, EarlyStopping) may be reused as training utilities. Your goal is to understand WHY recurrent networks behave as they do, not just to run the code.

Task 1: Text Loading, Cleaning, Tokenisation & Padding

#	Action	Details / Configuration
1.1	**Load and inspect**	df = pd.read_csv('IMDB Dataset.csv'). Print df.shape, df.head(), df['sentiment'].value_counts() and df.isnull().sum(). Print two raw reviews in full to see the HTML tags and punctuation that need cleaning.
1.2	**Remove duplicates**	Count duplicates with df.duplicated().sum(), then df.drop_duplicates(inplace=True). Report rows before and after. Markdown: WHY duplicates must be removed BEFORE the train/test split — otherwise the same review can appear in both sets and inflate test accuracy (data leakage).
1.3	**Clean the text**	Write clean_text(s): lowercase, remove HTML tags with re.sub(r'<.*?>', ' ', s), remove characters other than letters and apostrophes, collapse repeated spaces. Apply to the review column. Print one review before and after cleaning. Markdown: WHY apostrophes are kept (do not, can not) and why stop-words are NOT removed for sequence models (words like 'not' and 'no' flip sentiment and word order matters).
1.4	**Review length analysis**	Compute word count per review. Plot a histogram of lengths with vertical lines at the mean, median and 95th percentile. Plot a boxplot of length by sentiment. Markdown: describe the distribution and justify the padding length you will choose in 1.6 (use roughly the 90th-95th percentile, rounded).
1.5	**Tokenise**	from tensorflow.keras.preprocessing.text import Tokenizer. tokenizer = Tokenizer(num_words=10000, oov_token='<OOV>'). Fit on the TRAINING texts only. Convert texts to integer sequences with texts_to_sequences. Print the 10 most frequent words from tokenizer.word_index and one review as its integer sequence. Markdown: WHY num_words=10000 (rare words add parameters and noise) and what the OOV token does.
1.6	**Pad / truncate and split**	Use train_test_split(test_size=0.2, random_state=42, stratify=y) on the cleaned texts BEFORE fitting the tokenizer. Then pad_sequences(seqs, maxlen=MAXLEN, padding='pre', truncating='pre'). Print the final shapes: X_train (n, MAXLEN), y_train (n,). Markdown: WHY pre-padding is preferred for RNNs that read the LAST time step (the network's final hidden state should be built from real words, not from a long run of zeros).
1.7	**Sequence vs bag-of-words — Markdown**	Add a Markdown cell explaining why this data is SEQUENTIAL: the sentences 'the movie was not good' and 'the movie was good, not bad' contain nearly the same words but opposite meaning. A model that ignores order cannot separate them. This motivates Task 2.

Task 2: RNN Intuition, RNN vs ANN & Forward Propagation

#	Action	Details / Configuration
2.1	**Baseline ANN on text**	Build the order-blind baseline: Embedding(10000, 32, input_length=MAXLEN) → Flatten → Dense(64, relu) → Dense(1, sigmoid). Compile with Adam and binary_crossentropy. Train 5 epochs, validation_split=0.1. Print model.summary(). Record test accuracy and the parameter count. Markdown: WHY the ANN treats each position as an independent input slot and cannot share what it learns about a word across positions; note how its parameter count grows with MAXLEN.
2.2	**RNN intuition — Markdown**	Add a Markdown cell with the core idea: an RNN reads one word at a time and carries a hidden state h(t) that summarises everything read so far. The SAME weights are reused at every time step (weight sharing across time), so the model handles any sequence length with a fixed number of parameters. Include the equation h(t) = tanh(Wxh · x(t) + Whh · h(t-1) + b) and explain each term, plus the output y(t) = f(Why · h(t) + by).
2.3	**Manual forward propagation in NumPy**	Implement a single RNN cell from scratch with NumPy using hidden size 3 and input size 2. Fix the weights: Wxh (3×2), Whh (3×3), b (3,), all with np.random.seed(42) and np.random.randn * 0.5. Feed a 4-step input sequence of shape (4, 2). For each step compute h(t) = np.tanh(Wxh @ x(t) + Whh @ h(t-1) + b) starting from h(0) = zeros. Print every h(t) and display them as a small heatmap (time steps on x-axis, hidden units on y-axis).
2.4	**Verify against Keras**	Build keras SimpleRNN(3, return_sequences=True, use_bias=True) on input shape (4, 2). Copy your NumPy weights into it using layer.set_weights([Wxh.T, Whh.T, b]) (Keras stores kernels as input×units). Run the same sequence and confirm with np.allclose that Keras output equals your manual h(t) values. Markdown: explain any transposition needed.
2.5	**Parameter count by formula**	For SimpleRNN(units=64) on embedding size 32, the parameter count is units × (embedding_dim + units + 1) = 64 × (32 + 64 + 1) = 6,208. Verify this against model.summary() for your own model, and explain each of the three terms (input weights, recurrent weights, bias). Markdown: note that this count does NOT depend on MAXLEN, unlike the ANN baseline in 2.1.
2.6	**RNN vs ANN summary table — Markdown**	Add a Markdown table comparing ANN and RNN on: input handling, word order awareness, weight sharing, parameters vs sequence length, memory of earlier words, and typical use cases (tabular data vs text, speech, time series).

Task 3: Types of RNN & Sentiment Analysis with SimpleRNN

#	Action	Details / Configuration
3.1	**Types of RNN — shape demonstration**	Using small dummy tensors (batch 4, 10 time steps, 8 features), build and print the output shape for each type: (a) One-to-One: Dense(1) on a single vector (ordinary ANN). (b) One-to-Many: Input(8) → RepeatVector(10) → SimpleRNN(16, return_sequences=True). (c) Many-to-One: SimpleRNN(16) with return_sequences=False — output (4, 16). (d) Many-to-Many: SimpleRNN(16, return_sequences=True) — output (4, 10, 16). Present all four in a Markdown table with: type, input shape, output shape, return_sequences setting and an example application.

3.2	**Match types to real problems — Markdown**	For each of the four types give one concrete example (for instance image captioning is one-to-many, sentiment analysis is many-to-one, named-entity tagging and machine translation are many-to-many). State clearly which type this project uses and WHY (a whole review maps to one label).
3.3	**Build the SimpleRNN sentiment model**	build_model('rnn', units=64): Embedding(10000, 32) → SimpleRNN(64) → Dense(1, sigmoid). Compile with Adam (lr=0.001) and binary_crossentropy. Print model.summary() and verify the SimpleRNN parameter count with the formula from Task 2.5.
3.4	**Train and plot**	Train for up to 15 epochs, batch_size=64, validation_split=0.1, with EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True). Plot a 1×2 figure of training vs validation loss and accuracy. Record time per epoch (use a custom callback or time.time()). Markdown: describe the curves honestly — does the SimpleRNN learn smoothly or stay near 50-60% for several epochs?
3.5	**Evaluate**	Compute test accuracy, classification_report, confusion matrix heatmap and ROC-AUC. Store all metrics in a Python dictionary results['SimpleRNN'] — you will reuse it in Task 7.
3.6	**Embedding layer — Markdown**	Explain what the Embedding layer does: it turns each integer word ID into a learned dense vector, so similar words can end up with similar vectors. Note that these embeddings are trained from scratch with the model (no pretrained vectors in this practical). Print the shape of embedding weights (10000, 32) and show the embedding vectors of two words (for example 'good' and 'bad') using tokenizer.word_index.

Task 4: Backpropagation Through Time & Problems with RNN

#	Action	Details / Configuration
4.1	**BPTT — Markdown**	Write a clear Markdown explanation of Backpropagation Through Time: (1) the RNN is unrolled across T time steps into a deep network whose layers share weights; (2) the loss at the last step sends gradients backwards through every time step; (3) the gradient of the loss w.r.t. an early hidden state is a PRODUCT of T Jacobian terms, each containing Whh and the tanh derivative; (4) the gradient w.r.t. a shared weight is the SUM of its contributions from every time step. Include the chain: dL/dh(1) = dL/dh(T) × ∏ (Whh^T · diag(1 − h(t)^2)).
4.2	**Vanishing and exploding gradients — Markdown**	Explain: if the repeated factor has magnitude below 1 (tanh derivative is at most 1, usually much smaller), the product shrinks exponentially and early words stop influencing learning — the vanishing gradient problem. If the factor exceeds 1 (large Whh values), the product grows exponentially — the exploding gradient problem, which shows up as NaN loss or wild loss spikes. State the practical consequence: a SimpleRNN has a short effective memory of roughly 10-20 time steps.
4.3	**Measure gradient flow with GradientTape**	Build a model that exposes hidden states: Embedding → SimpleRNN(64, return_sequences=True) → take the LAST time step → Dense(1, sigmoid). Pass one batch (64 reviews) through it inside tf.GradientTape. Watch the SimpleRNN output tensor H (shape batch × T × 64) with tape.watch(H), compute binary cross-entropy on the final prediction, and get grads = tape.gradient(loss, H). Compute the norm of the gradient at each time step: tf.norm(grads, axis=-1) averaged over the batch. Plot gradient norm (log y-axis) against time step (1 to MAXLEN). Title: 'Gradient Norm per Time Step — SimpleRNN'. You should see the norm collapse by orders of magnitude toward early time steps.

4.4	**Accuracy vs sequence length**	Train the same SimpleRNN model at four padding lengths: 50, 100, 200 and 300 (re-pad the data each time, 8 epochs each, same seed). Plot test accuracy against sequence length. Markdown: explain why accuracy does not improve (and may even fall) as sequences get longer — the network cannot carry information across that many steps, so extra words add noise rather than signal.
4.5	**Exploding gradient demo and clipping**	Train the SimpleRNN with a deliberately large learning rate (for example Adam lr=0.05 or SGD lr=1.0) for 5 epochs and plot the loss — note spikes or NaN. Retrain with the same optimiser plus gradient clipping: clipnorm=1.0. Plot both loss curves on the same axes. Markdown: explain that clipping rescales the gradient vector when its norm exceeds the threshold, fixing EXPLODING gradients but doing nothing for VANISHING gradients — which is why gated architectures were invented.
4.6	**Problems summary table — Markdown**	Add a Markdown table: Problem | Cause | Symptom you observed in this notebook | Remedy. Rows: vanishing gradient (remedy: LSTM / GRU gating), exploding gradient (remedy: gradient clipping), slow sequential training (remedy: smaller sequences, GPU), short-term memory only (remedy: gated cells).

Task 5: LSTM — Long Short-Term Memory

#	Action	Details / Configuration
5.1	**LSTM theory — Markdown**	Explain the LSTM cell with the four components and their equations: cell state C(t) (the 'conveyor belt' that carries information with only minor linear changes), forget gate f(t) = sigmoid(Wf·[h(t-1), x(t)] + bf), input gate i(t) = sigmoid(Wi·[h(t-1), x(t)] + bi), candidate state g(t) = tanh(Wg·[h(t-1), x(t)] + bg), cell update C(t) = f(t)*C(t-1) + i(t)*g(t), output gate o(t) = sigmoid(Wo·[h(t-1), x(t)] + bo), hidden state h(t) = o(t)*tanh(C(t)). Explain in plain words what each gate decides (what to forget, what to write, what to reveal).
5.2	**WHY LSTM fixes vanishing gradients — Markdown**	Explain that the cell state is updated by addition and element-wise multiplication with the forget gate, so the gradient along C(t) is multiplied by f(t) (close to 1 when the gate stays open) rather than by a weight matrix and a tanh derivative at every step. This gives the gradient a near-uninterrupted path across many time steps.
5.3	**Parameter count by formula**	An LSTM has four weight sets (forget, input, candidate, output). Verify: parameters = 4 × units × (embedding_dim + units + 1). For LSTM(64) on embedding 32: 4 × 64 × (32 + 64 + 1) = 24,832. Confirm against model.summary() and explain why this is exactly 4 times the SimpleRNN count.
5.4	**Manual single-step LSTM in NumPy**	Using hidden size 3 and input size 2, implement one LSTM step with NumPy: compute f, i, g, o, then C(t) and h(t). Use random weights with a fixed seed. Run 4 time steps and print f, i, o, C(t) and h(t) at each step as a small table. Markdown: describe what happens to C(t) when f is close to 1 and i is close to 0 (the memory is preserved unchanged).
5.5	**Build, train and evaluate the LSTM**	build_model('lstm', units=64): Embedding(10000, 32) → LSTM(64) → Dense(1, sigmoid). Same optimiser, batch size, EarlyStopping and epochs as Task 3.4 so the comparison is fair. Plot loss/accuracy curves, record time per epoch, and compute test accuracy, classification_report, confusion matrix and ROC-AUC. Store in results['LSTM'].
5.6	**Gradient flow comparison**	Repeat the GradientTape analysis from Task 4.3 on the LSTM (return_sequences=True, then last step to Dense). Plot the

		per-time-step gradient norm of the LSTM on the SAME axes as the SimpleRNN from Task 4.3 (log y-axis). Markdown: state quantitatively how much larger the LSTM gradient is at the earliest time steps and relate it to the gating explanation in 5.2.
5.7	**Stacked LSTM**	Build a 2-layer LSTM: Embedding → LSTM(64, return_sequences=True) → LSTM(32) → Dense(1, sigmoid). Explain why the first layer needs return_sequences=True (the second LSTM needs a full sequence as input). Train, evaluate and add to results['Stacked LSTM']. Markdown: does the extra depth help on this dataset, and what does it cost in parameters and training time?

Task 6: GRU — Gated Recurrent Unit

#	Action	Details / Configuration
6.1	**GRU theory — Markdown**	Explain the GRU as a simplified gated cell with TWO gates and no separate cell state: update gate z(t) = sigmoid(Wz·[h(t-1), x(t)]), reset gate r(t) = sigmoid(Wr·[h(t-1), x(t)]), candidate h~(t) = tanh(W·[r(t)*h(t-1), x(t)]), and new state h(t) = (1 − z(t))*h(t-1) + z(t)*h~(t). Explain in plain words: the update gate decides how much of the old state to keep versus replace; the reset gate decides how much of the past to use when proposing the new candidate.
6.2	**GRU vs LSTM — Markdown table**	Compare GRU and LSTM on: number of gates, separate cell state (yes/no), parameters for the same hidden size, training speed, and typical situations where each is preferred. State clearly that GRU merges the forget and input gates into one update gate.
6.3	**Parameter count by formula**	A GRU has three weight sets. With Keras default reset_after=True there are two bias vectors per set, so parameters = 3 × (units × embedding_dim + units × units + 2 × units). For GRU(64) on embedding 32: 3 × (2,048 + 4,096 + 128) = 18,816. Confirm against model.summary(), and explain why the count is about three quarters of the LSTM count.
6.4	**Build, train and evaluate the GRU**	build_model('gru', units=64): Embedding(10000, 32) → GRU(64) → Dense(1, sigmoid). Identical training setup to Tasks 3.4 and 5.5. Plot curves, record time per epoch, compute test accuracy, classification_report, confusion matrix and ROC-AUC. Store in results['GRU'].
6.5	**Gradient flow — all three cells**	Extend the gradient-norm plot from Task 5.6 to include the GRU: one figure with SimpleRNN, LSTM and GRU curves (log y-axis). Markdown: rank the three cells by how well gradients reach early time steps and explain the ranking.
6.6	**Hidden-unit sensitivity**	Train LSTM and GRU with units = 32, 64 and 128 (6 epochs each). Plot test accuracy against units for both cells on the same axes, and also plot parameter count against units. Markdown: identify the point of diminishing returns.

Task 7: Final Comparison, Inference & Recommendation

#	Action	Details / Configuration
7.1	**Full results table**	Build a pandas DataFrame from the results dictionary with one row per model: Model | Parameters | Time per Epoch (s) | Epochs Trained | Test

		Accuracy | Precision | Recall | F1 | ROC-AUC. Rows: ANN baseline (Task 2.1), SimpleRNN, LSTM, Stacked LSTM, GRU. Highlight the best value in each metric column with df.style.highlight_max(color='#ffcccc'). Save the table as an image or CSV into /plots.
7.2	**ROC curves**	Plot ROC curves for ANN baseline, SimpleRNN, LSTM and GRU on one axes with AUC values in the legend. Markdown: what does the ROC curve show that accuracy alone does not?
7.3	**Sequence-length robustness**	Take your best LSTM or GRU model and evaluate its test accuracy separately on SHORT reviews (below 100 words), MEDIUM (100-250) and LONG (above 250). Present as a grouped bar chart together with the SimpleRNN for comparison. Markdown: which model degrades most on long reviews and why?
7.4	**Error analysis**	For your best model, print 8 misclassified test reviews (4 false positives, 4 false negatives) with the true label, the predicted probability and the cleaned text. Markdown: identify patterns — sarcasm, negation, mixed reviews, very long reviews, reviews describing a 'bad' character in a 'good' film.
7.5	**Inference function**	Write predict_sentiment(text) that applies the SAME clean_text, tokenizer and padding as training, then returns the label and probability from your best model. Test it on at least 6 custom sentences you write yourself, including: one clearly positive, one clearly negative, one with negation ('this film is not good'), one with double negation, one sarcastic, and one very short sentence. Display results in a table and comment on which ones the model gets wrong and why.
7.6	**Recommendation — Markdown**	Write a section titled 'Which Recurrent Cell Should We Use?'. Cover: (a) accuracy, F1 and ROC-AUC ranking; (b) training time and parameter trade-off; (c) behaviour on long sequences; (d) your final recommendation for a production sentiment service and WHY; (e) two limitations of the whole approach (for example no pretrained knowledge, one-directional reading) stated as observations only — do not implement them.
7.7	**Notebook quality**	Every task has an opening Markdown cell. All plots have titles, axis labels and legends. Set random seeds (np.random.seed, tf.random.set_seed) at the top for reproducibility. Run Kernel > Restart & Run All with zero errors. Export to HTML. Create requirements.txt listing tensorflow, scikit-learn, pandas, numpy, matplotlib, seaborn.

§ 3 · Transparent Marking Scheme (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	5 Marks	50%	Completed notebook — all 7 tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + .html + plots + README + video link
TOTAL	**10 Marks**	**100%**	

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**0.5**	CSV loaded; duplicates removed with leakage explanation; clean_text removes HTML, punctuation and lowercases; length histogram with mean/median/95th percentile and boxplot by sentiment; Tokenizer fitted on training set only with OOV token; split done BEFORE tokenisation; pre-padding with justified MAXLEN; sequential-data Markdown present	Cleaned, tokenised and padded correctly; EDA or Markdown partly missing	Tokenised and padded; no length analysis or leakage discussion	CSV loaded; text not cleaned or tokenised properly	Not done
A2	**1.0**	Order-blind ANN baseline trained and parameter count noted; RNN intuition Markdown with equation;	ANN baseline + manual forward pass + Keras verification done; types	ANN baseline and Markdown only; manual forward pass missing or not verified	Only a SimpleRNN built without foundation tasks	None

		manual NumPy forward pass over 4 steps with heatmap; verified against Keras with np.allclose; parameter formula checked; RNN vs ANN table; all four RNN types shown with correct output shapes and examples	of RNN table or formula check incomplete			
A3	**1.0**	SimpleRNN sentiment model trained with curves, time per epoch, full metrics; BPTT and vanishing/exploding gradient Markdown; GradientTape per-time-step gradient norm plot; accuracy vs sequence length experiment (4 lengths); exploding gradient demo with and without clipnorm; problems summary table	SimpleRNN trained and evaluated; gradient-norm plot and one other experiment completed; Markdown partly complete	SimpleRNN trained; no gradient analysis or sequence-length experiment	SimpleRNN built but training or evaluation incomplete	None
A4	**1.5**	LSTM theory with all gate equations; vanishing-gradient fix explained; parameter formula verified (4×); manual NumPy LSTM step; LSTM trained and fully evaluated; gradient comparison vs SimpleRNN; stacked LSTM with return_sequences explained; GRU theory with both gates; GRU vs LSTM	LSTM and GRU both trained and evaluated with theory; one of manual LSTM step, stacked LSTM or sensitivity study missing	LSTM and GRU trained; theory Markdown thin; no gradient comparison	Only one of LSTM or GRU implemented properly	None

		table; GRU parameter formula (reset_after) verified; GRU trained and evaluated; 3-way gradient plot; hidden-unit sensitivity study				
A5	**1.0**	Results table for 5 models with parameters, time per epoch and 5 metrics, best highlighted; combined ROC plot; accuracy by review length chart; 8 misclassified reviews analysed; predict_sentiment() reusing exact training preprocessing and tested on 6+ custom sentences including negation and sarcasm; recommendation section; seeds set; Restart & Run All clean	Results table, ROC and inference function complete; error analysis or length-robustness missing	Results table present; no inference function or error analysis	Partial table only	None

Subtotal A: 0.5 + 1.0 + 1.0 + 1.5 + 1.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook screen. Explain concepts verbally — do not just run cells silently.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video; face visible via PiP throughout; plots, model summaries and result tables clearly visible; all 7 tasks	Clear audio; face mostly visible; most tasks shown	Audio unclear; face partially visible	Very poor quality, hard to follow	No video

		demonstrated in order				
B2	**1.5**	Verbally explains: (1) WHY order matters in text and why the ANN baseline fails to use it; (2) the RNN equation, hidden state and weight sharing across time, using your NumPy forward pass; (3) the four types of RNN and which one sentiment analysis uses; (4) BPTT and WHY the gradient is a product of many terms; (5) vanishing versus exploding gradient and WHY clipping fixes only one; (6) LSTM gates and the cell state as a protected memory path; (7) GRU update and reset gates and how GRU differs from LSTM; (8) your final recommendation and what the gradient-norm plots proved	Explains 6–7 concepts with reasoning	Demonstrates steps without explaining theory	Reads from cells; no analysis	No verbal explanation
B3	**0.5**	5–10 min; shows manual forward-pass heatmap, gradient-norm plot, all three training curves, results table and a live predict_sentiment() demo with a new sentence	4–5 min; most tasks covered	2–4 min; key experiments missing	Under 2 min or very incomplete	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Video Requirements

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Jupyter Notebook — manual forward pass, gradient-norm plots, training curves, results table, live inference demo
Duration	5 to 10 minutes
Naming convention	DL_PR5_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (2 Marks)

Create a public GitHub repository specifically for this project.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo contains: DL_PR5.ipynb + DL_PR5.html + /plots with clearly named figures (length_distribution.png, rnn_forward_heatmap.png, gradient_norms_all_cells.png, training_curves_rnn_lstm_gru.png, roc_curves.png, results_table.png, accuracy_by_length.png) + requirements.txt	Key files present; a few plots missing	Only .ipynb uploaded; no plots	Repo exists but incomplete or broken	No repo
C2	**1.0**	README: title + Kaggle dataset link and citation of Maas et al. + preprocessing summary + one paragraph each on RNN, LSTM and GRU + gradient-norm plot with	Title + dataset + partial explanations and plots	Title and dataset only	Blank README or single line	No README

		caption + full results table + recommendation + video link + tools used				
C3	**0.5**	3+ meaningful commits, e.g. 'Add text cleaning, tokenisation and padding pipeline'; 'Add manual RNN forward pass and BPTT gradient-norm analysis'; 'Add LSTM and GRU models with comparison table and inference function'	2–3 adequate commits	1 bulk commit with all files	Unclear or auto-generated messages	No commits

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

Faculty Marking Sheet

Component	Criterion	Max	Score	Remarks
A (5)	A1. Text Cleaning, Tokenisation, Padding & Split	0.5		
	A2. RNN Intuition, ANN Baseline, Manual Forward Pass, Types of RNN	1.0		
	A3. SimpleRNN Sentiment Model, BPTT & Gradient Experiments	1.0		
	A4. LSTM and GRU — Theory, Models, Gradient Comparison	1.5		
	A5. Results Table, ROC, Error Analysis, Inference, Recommendation	1.0		
B (3)	B1. Video Quality (Audio, Face PiP, Screen Clarity)	1.0		
	B2. Verbal Explanation of 8 Core Concepts	1.5		
	B3. Duration & Completeness (5–10 min, live inference demo)	0.5		
C (2)	C1. Repo Structure (.ipynb + .html + /plots + requirements.txt)	0.5		

	C2. README.md Quality	1.0		
	C3. Commit History (3+ meaningful commits)	0.5		
TOTAL		**10.0**		

§ 4 · Submission Checklist

Tick every item below before submitting. All items are required for full marks.

- ☐ IMDB Dataset.csv downloaded from the Kaggle URL in § 1 (NOT keras.datasets.imdb)
- ☐ Shape, class balance and null check printed; two raw reviews displayed
- ☐ Duplicates counted and removed; Markdown on data leakage
- ☐ clean_text() removes HTML tags, non-letter characters and lowercases; before/after example shown
- ☐ Markdown: why stop-words are kept and apostrophes preserved
- ☐ Review length histogram (mean, median, 95th percentile lines) and boxplot by sentiment
- ☐ Train/test split (80/20, stratified, random_state=42) done BEFORE fitting the tokenizer
- ☐ Tokenizer(num_words=10000, oov_token) fitted on training texts only; top-10 words printed
- ☐ pad_sequences with justified MAXLEN and padding='pre'; final shapes printed
- ☐ Markdown: why text is sequential ('not good' vs 'good, not bad')
- ☐ Order-blind ANN baseline (Embedding → Flatten → Dense) trained; parameters and test accuracy recorded
- ☐ RNN intuition Markdown with h(t) equation and weight sharing across time
- ☐ Manual NumPy RNN forward pass (4 steps, hidden size 3) with hidden-state heatmap
- ☐ Manual forward pass verified against Keras SimpleRNN using np.allclose
- ☐ SimpleRNN parameter formula units × (input + units + 1) verified against model.summary()
- ☐ RNN vs ANN comparison table
- ☐ Four types of RNN built on dummy data with output shapes printed; table with examples; project type justified
- ☐ SimpleRNN sentiment model trained with EarlyStopping; curves plotted; time per epoch recorded
- ☐ SimpleRNN evaluated: accuracy, classification_report, confusion matrix, ROC-AUC stored in results
- ☐ Embedding layer Markdown and embedding vectors of two words shown
- ☐ BPTT Markdown including the product-of-Jacobians chain
- ☐ Vanishing and exploding gradient Markdown
- ☐ GradientTape per-time-step gradient norm plot (log scale) for SimpleRNN
- ☐ Accuracy vs sequence length experiment (50, 100, 200, 300) plotted with explanation
- ☐ Exploding gradient demo with high learning rate; same run repeated with clipnorm=1.0; both losses plotted
- ☐ Problems summary table (problem, cause, symptom observed, remedy)
- ☐ LSTM theory Markdown with all gate and state equations
- ☐ Markdown: why the cell state mitigates vanishing gradients
- ☐ LSTM parameter formula 4 × units × (input + units + 1) verified
- ☐ Manual NumPy LSTM step with f, i, o, C(t), h(t) printed over 4 steps
- ☐ LSTM trained with identical setup to SimpleRNN; fully evaluated and stored in results
- ☐ Gradient-norm plot: LSTM vs SimpleRNN on the same axes
- ☐ Stacked LSTM (return_sequences=True on first layer) trained and evaluated
- ☐ GRU theory Markdown with update and reset gate equations
- ☐ GRU vs LSTM comparison table
- ☐ GRU parameter formula (reset_after=True) verified against model.summary()
- ☐ GRU trained with identical setup; fully evaluated and stored in results

- ☐ Gradient-norm plot with SimpleRNN, LSTM and GRU together; ranking explained
- ☐ Hidden-unit sensitivity (32, 64, 128) for LSTM and GRU: accuracy and parameter plots
- ☐ Results table for 5 models (parameters, time per epoch, epochs, accuracy, precision, recall, F1, ROC-AUC) with best values highlighted
- ☐ Combined ROC curve for ANN, SimpleRNN, LSTM, GRU with AUC in legend
- ☐ Accuracy by review length (short / medium / long) grouped bar chart
- ☐ Eight misclassified reviews analysed (4 false positives, 4 false negatives) with pattern discussion
- ☐ predict_sentiment(text) reuses exact training preprocessing; tested on 6+ custom sentences including negation and sarcasm
- ☐ Recommendation section 'Which Recurrent Cell Should We Use?' written
- ☐ Random seeds set; every task begins with an opening Markdown cell
- ☐ All plots have titles, axis labels and legends
- ☐ Kernel > Restart & Run All completed with zero errors; notebook exported to HTML
- ☐ requirements.txt committed (tensorflow, scikit-learn, pandas, numpy, matplotlib, seaborn)
- ☐ Video: face (PiP) + Jupyter screen, 5–10 min, MP4/MOV, named DL_PR5_YourName_GRID.mp4
- ☐ Video explains all 8 core concepts and shows a live predict_sentiment() demo
- ☐ GitHub: public repo with DL_PR5.ipynb + DL_PR5.html + /plots + requirements.txt
- ☐ README.md: dataset link and citation, preprocessing summary, RNN / LSTM / GRU paragraphs, gradient plot, results table, recommendation, video link, tools used
- ☐ 3+ meaningful Git commits with descriptive messages
- ☐ Video link pasted in README.md under the Video section

© 2026 Red & White Skill Education. All Rights Reserved.